Django & DRF with Rattan - Complete Course Notes

▼ Part 1

Part 1: Django Fundamentals & Project Setup

1. What is Django?

Definition: Django is a high-level Python web framework that enables rapid development of secure and maintainable websites.

Key Points:

- Follows MVT (Model-View-Template) architecture
- Comes with built-in admin panel
- Includes ORM (Object-Relational Mapping)
- Battery-included philosophy - provides authentication, sessions, messages, etc. out of the box

Why Django?

- Fast development
- Less code required
- Built-in security features
- Scalable for large applications

2. MVT Architecture Explained

MVT = Model-View-Template

Component	Responsibility
Model	Database operations, data structure
View	Business logic, request handling
Template	Presentation layer (HTML)

Request Flow:

```
User → URL → View → Model (if DB needed) → View → Template → Response to User
```

Important: Model never directly talks to Template or User. View acts as the mediator.

---

3. Creating Virtual Environment

Why Virtual Environment?

- Isolates project dependencies
- Prevents package conflicts between projects
- Keeps global Python clean

Commands:

```
# dir      #show all directoires/folders only
# ls       #show all files and folder in current dir
# pip freeze   #show all install pkgs in current venv
# Create virtual environment
python -m venv env          # uv venv

# Activate (Windows)
env\Scripts\activate

# Activate (gitbash)
source env/Scripts/activate

# Deactivate
deactivate
```

Verify Activation: You'll see `(env)` prefix in terminal

---

4. Installing Django & Creating Project

Install Django:

```
pip install django
```

Create Project:

```
django-admin startproject project_name .  # create project
python manage.py startapp app_name  # create app inside project
```

Note: The `.` at the end creates project in current directory (recommended)

Project Structure:

```
project_name/
├── manage.py
├── db.sqlite3
├── project_name/
│   ├── __init__.py
│   ├── settings.py
│   ├── urls.py
│   ├── asgi.py
│   └── wsgi.py
```

---

5. Important Files Explained

manage.py:

- Entry point for all Django commands
- Like a "receptionist" - you tell it what to do, it forwards to Django

settings.py:

- All project configurations
- Database settings, installed apps, middleware, templates, etc.

urls.py:

- URL routing configuration
- Maps URLs to views

wsgi.py/asgi.py:

- HTTP server configurations
- Used in production deployment

---

6. Initial Setup Commands

```
# Apply migrations (create default tables)
python manage.py migrate
```

```
# Create superuser (admin)
python manage.py createsuperuser

# Run development server
python manage.py run server
```

Default server URL: `http://127.0.0.1:8000/`

7. Key settings.py Configurations

BASE_DIR:

```
BASE_DIR = Path(__file__).resolve().parent.parent
```

Dynamic path to project root - works on any system

SECRET_KEY:

- Used for signing sessions, tokens, etc.
- Must be kept secret in production
- Use environment variables in production

DEBUG:

```
DEBUG = True  # Development only
DEBUG = False # Production
```

- `True`: Shows detailed error pages
- `False`: Shows standard 404/500 pages

ALLOWED_HOSTS:

```
ALLOWED_HOSTS = []  # Empty in development
ALLOWED_HOSTS = ['yourdomain.com', '192.168.1.1']  # Production
```

INSTALLED_APPS:

```
INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
```

```
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    # Your apps here
]
```

8. Creating Django Apps

```
python manage.py start app app_name
```

App Structure:

```
app_name/
├── __init__.py
├── admin.py
├── apps.py
├── models.py
├── views.py
├── urls.py (create manually)
├── tests.py
└── migrations/
```

Register App in settings.py:

```
INSTALLED_APPS = [
    # ...
    'app_name',
]
```

9. Models - Database Tables

Creating a Model:

```
from django.db import models

class Department(models.Model):
    name = models.CharField(max_length=25)
    location = models.CharField(max_length=25)
```

```
    def __str__(self):
        return f"{self.name} - {self.location}"

class Employee(models.Model):
    first_name = models.CharField(max_length=50)
    last_name = models.CharField(max_length=50)
    email = models.EmailField(unique=True)
    designation = models.CharField(max_length=50)
    salary = models.IntegerField()
    joining_date = models.DateField()
    is_active = models.BooleanField(default=True)
    image = models.ImageField(upload_to='employees/', default='defaul
t.png')

    # Foreign Key (Many-to-One)
    department = models.ForeignKey(
        'Department',
        on_delete=models.SET_NULL,
        null=True,
        blank=True
    )

    # Self-referencing Foreign Key
    manager = models.ForeignKey(
        'self',
        on_delete=models.SET_NULL,
        null=True,
        blank=True
    )

    def __str__(self):   # member function
        return f"{self.first_name} {self.last_name}"

    def full_name(self):  # virtual field
            return f"{self.first_name} {self.last_name}"
```

Common Field Types:

Field Type	Use Case
`CharField`	Short text (requires max_length)
`TextField`	Long text
`IntegerField`	Numbers
`EmailField`	Email addresses
`DateField`	Dates
`BooleanField`	True/False
`ImageField`	Images (requires Pillow)
`ForeignKey`	Many-to-One relationship

Field Options:

Option	Purpose
`max_length`	Maximum characters
`unique=True`	No duplicates allowed
`null=True`	Allow NULL in database
`blank=True`	Allow empty in forms
`default`	Default value
`on_delete`	What happens when referenced object is deleted

on_delete Options:

- `CASCADE` - Delete this too
- `SET_NULL` - Set to NULL (requires null=True)
- `SET_DEFAULT` - Set to default value
- `PROTECT` - Prevent deletion

---

10. Migrations

What are Migrations?

- Instructions for database to create/modify tables
- Like "blueprints" for your database
- when you change the model you have to run migrations, not when you change in method / member functions.

Commands:

```
# Create migration files (after model changes)
python manage.py makemigrations

# Apply migrations to database
python manage.py migrate

# See migration status
python manage.py showmigrations
```

Flow:

```
Model Changes → makemigrations → Migration File → migrate → Database Updated
```

Important: Never edit migration files manually!

11. Admin Panel Registration

```
# admin.py
from django.contrib import admin
from .models import Employee, Department

# Customized registration
class EmployeeAdmin(admin.ModelAdmin):
    list_display = ('full_name', 'email', 'manager', 'salary')
    list_filter = ('department', 'is_active')
    search_fields = ( 'full_name', 'email')
    list_editable = ('salary',)
    ordering = ('-id',)

# Basic registration
admin.site.register(Employee, EmployeeAdmin)
admin.site.register(Department)
```

Admin Customization Options:

Option	Purpose
`list_display`	Columns to show in list

Option	Purpose
`list_filter`	Sidebar filters
`search_fields`	Searchable fields
`list_editable`	Editable from list view
`ordering`	Default ordering
`list_display_links`	Clickable columns

12. Virtual Fields in Models

Creating computed fields:

```
class Employee(models.Model):
    # ... fields ...

    def full_name(self):
        return f"{self.first_name} {self.last_name}"
```

Usage in Admin:

```
class EmployeeAdmin(admin.ModelAdmin):
    list_display = ('full_name', 'email')
```

13. URLs Configuration

Project-level urls.py:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('', include('employees.urls')),
]
```

App-level urls.py:

```
from django.urls import path
from . import views
```

```
urlpatterns = [
    path('', views.home, name='home'),
    path('add/', views.add_employee, name='add_employee'),
    path('<int:id>/', views.employee_detail, name='employee_detail'),
]
```

URL Pattern Types:

```
path('employees/', view_name)           # Static
path('employees/<int:id>/', view_name)  # Integer parameter
path('employees/<str:name>/', view_name) # String parameter
path('', include('app.urls'))           # Include app URLs
```

14. Views - Function Based

```
from django.shortcuts import render, redirect, get_object_or_404
from django.http import HttpResponse
from .models import Employee

def home(request):
    employees = Employee.objects.all()
    context = {'employees': employees}
    return render(request, 'home.html', context)

def employee_detail(request, id):
    employee = get_object_or_404(Employee, pk=id)
    context = {'employee': employee}
    return render(request, 'employee_detail.html', context)
```

Request Methods:

```
def add_employee(request):
    if request.method == 'POST':
        # Handle form submission
        form = EmployeeForm(request.POST, request.FILES)
        if form.is_valid():
            form.save()
```

```
            return redirect('home')
    else:
        form = EmployeeForm()

    context = {'form': form}
    return render(request, 'add_employee.html', context)
```

15. Templates

Template Configuration in settings.py:

```
TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': ['templates'],  # Add this
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                # ...
            ],
        },
    },
]
```

Template Tags:

```
<!-- Variables -->
{{ variable }}

<!-- For Loop -->
{% for item in items %}
    {{ item.name }}
{% endfor %}

<!-- If Condition -->
{% if user.is_authenticated %}
    <p>Welcome!</p>
{% else %}
    <p>Please login</p>
```

```
{% endif %}

<!-- URL -->
<a href="{% url 'url_name' %}">Link</a>  <!--url_name in urls.py path name=root -->
<a href="{% url 'detail' item.id %}">Detail</a>

<!-- Load Static : at the top of html file -->
{% load static %}
<link rel="stylesheet" href="{% static 'css/style.css' %}">
<img src="{% static 'images/logo.png' %}">
```

16. Static Files

settings.py Configuration:

```
STATIC_URL = 'static/'
STATICFILES_DIRS = [BASE_DIR / 'static']
STATIC_ROOT = BASE_DIR / 'staticfiles'  # For production

# Create static folder in project root:
# ├── css/style.css
# ├── js/
# └── images/
```

Collect Static (Production only):

```
python manage.py collectstatic
```

17. Media Files (User Uploads)

settings.py Configuration:

```
MEDIA_URL = '/media/'
MEDIA_ROOT = BASE_DIR / 'media'
```

Project urls.py: // only store media in django while debug enabled - means in dev env

```
from django.conf import settings
from django.conf.urls.static import static

urlpatterns = [
    # ... your patterns
]

if settings.DEBUG: # cofig media url
    urlpatterns += static(settings.MEDIA_URL, document_root=settings.M
EDIA_ROOT)
```

Model with Image:

```
image = models.ImageField(upload_to='employees/', default='default.pn
g')
```

Template:

```
<img src="{{ employee.image.url }}" alt="{{ employee.first_name }}">
```

Form with File Upload:

```
<form method="POST" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}  <!-- render django form -->
    <button type="submit">Submit</button>
</form>
```

View with Files:

```
form = EmployeeForm(request.POST, request.FILES)
```

18. Django Forms

```
# forms.py in app i.e employees app
from django import forms
from .models import Employee
```

```
class EmployeeForm(forms.ModelForm):
    class Meta:
        model = Employee
        fields = '__all__'
        # or
        fields = ['first_name', 'last_name', 'email']

        widgets = {
            'joining_date': forms.DateInput(attrs={'type': 'date'}),
        }
```

Usage in View:

```
# path('add', views.add_employee, name="add_employee") # urls.py
def add_employee(request): #views.py
    if request.method == 'POST':
        form = EmployeeForm(request.POST, request.FILES)
        if form.is_valid():
            form.save()
            return redirect('home')
    else:
        form = EmployeeForm()
    return render(request, 'add.html', {'form': form})
```

template/add.html:

```
<form method="POST" enctype="multipart/form-data">
    {% csrf_token %}
    {{ form.as_p }}  <!-- render django form -->
    <button type="submit">Submit</button>
</form>
```

Bootstrap Crispy Forms in django: design ui of forms

i. install below core pakages

```
pip install django-crispy-forms crispy-bootstrap5
```

ii. Add both packages to your `INSTALLED_APPS` and define the required template pack settings:

```
INSTALLED_APPS = [
    # ... other apps
    "crispy_forms",
    "crispy_bootstrap5",
]

# Set Bootstrap 5 as the allowed and default styling
CRISPY_TEMPLATE_PACK = "bootstrap5"
```

iii. register.html

```
{% load static %}
{% load crispy_forms_tags %}     <---
<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=
1.0" />
    <link href="https://jsdelivr.net" rel="stylesheet" />    <---
    <link rel="stylesheet" href="{% static 'css/style.css' %}" />
    <title>Register Employee</title>
  </head>
  <body>
    <form method="POST" enctype="multipart/form-data">
      {% csrf_token %}
      {{ form | crispy }}         <---
      <button type="submit">Submit</button>
    </form>
  </body>
</html>
```

19. Authentication System

Django provides a built-in authentication system that handles users, groups, permissions, and sessions.

we create ‘acounts’ app and not create account or user model becuase user model is auto already created.

/register

/login

Built-in Forms:

```
from django.contrib.auth.forms import UserCreationForm, Authentication
Form
from django.contrib.auth import login, logout, authenticate
```

Registration View:

```
def register(request):
    if request.method == 'POST':
        form = UserCreationForm(request.POST)
        if form.is_valid():
            form.save()
            return redirect('login')
    else:
        form = UserCreationForm()
    return render(request, 'register.html', {'form': form})
```

Login View:

```
def login_view(request):
    if request.user.is_authenticated:
        return redirect('home')

    if request.method == 'POST':
        form = AuthenticationForm(request, data=request.POST)
        if form.is_valid():
            user = form.get_user()
            login(request, user)
            return redirect('home')
    else:
```

```
        form = AuthenticationForm()
    return render(request, 'login.html', {'form': form})
```

Logout View:

```
def logout_view(request):
    logout(request)
    return redirect('login')
```

20. Login Required Decorator

```
from django.contrib.auth.decorators import login_required

@login_required(login_url='login')
def add_employee(request):
    # Only accessible to logged-in users
    pass
```

settings.py:

```
LOGIN_URL = 'login'
```

21. Permissions & Groups

In Views:

```
from django.contrib.auth.decorators import permission_required

@permission_required('employees.add_employee', raise_exception=True)
def add_employee(request):
    pass
```

In Templates:

```
{% if perms.employees.add_employee %}
    <a href="{% url 'add_employee' %}">Add Employee</a>
{% endif %}
```

```
{% if perms.employees.change_employee %}
    <a href="{% url 'edit_employee' employee.id %}">Edit</a>
{% endif %}

{% if perms.employees.delete_employee %}
    <a href="{% url 'delete_employee' employee.id %}">Delete</a>
{% endif %}
```

Default Permissions (per model):

- `add_modelname`
- `change_modelname`
- `delete_modelname`
- `view_modelname`

example:

```
@login_required(login_url='login')
@permission_required("employee.add_employee", raise_exception=True)
def add_employee(request):
    if request.method == 'POST':
        form = EmployeeForm(request.POST, request.FILES)
        if form.is_valid():
            form.save()
            return redirect('home')
    else:
        form = EmployeeForm()
    return render(request, 'add_employee.html', {'form': form})
```

Groups

Groups are collections of permissions. Assign permissions to a group, then assign users to the group.

Admin Panel:

- Create groups (e.g., Manager, Editor)
- Assign permissions to groups
- Add users to groups

Programmatically:

```
from django.contrib.auth.models import Group, Permission

# Create group
manager_group, created = Group.objects.get_or_create(name='Manager')

# Add permissions
permissions = Permission.objects.filter(codename__in=[
    'add_employee', 'change_employee', 'view_employee'
])
manager_group.permissions.set(permissions)

# Assign user to group
user.groups.add(manager_group)
```

---

22. Context Processors

What it does: Makes variables available to ALL templates automatically

Built-in (in settings.py):

```
'context_processors': [
    'django.template.context_processors.request',
    'django.contrib.auth.context_processors.auth',
    'django.contrib.messages.context_processors.messages',
]
```

Usage in any template:

```
{% if user.is_authenticated %}
    <p>Hello, {{ user.username }}</p>
{% endif %}
```

---

23. Error Pages

Create templates:

- `templates/404.html`

- `templates/403.html`
- `templates/500.html`

Django automatically uses these when `DEBUG = False`

```
<!-- 404.html -->
<h1>Page Not Found</h1>
<p>The page you're looking for doesn't exist.</p>

<!-- 403.html -->
<h1>Access Denied</h1>
<p>You don't have permission to access this page.</p>
```

Design Admin Dashborad Design: use tool 'jazzmin' to change look of admin dashborad.

Django AWS Deployment Tips: EC2, AWS Elastic Beanstalk, AWS RDS Postgres, S3 bucket. note: until your app is not big for bussiness pespective, i'm not recomend aws.

Summary Cheat Sheet

```
# Virtual Environment
python -m venv env
source env/bin/activate  # Mac/Linux
env\Scripts\activate     # Windows

# Django Commands
pip install django
django-admin startproject project_name .
python manage.py startapp app_name
python manage.py makemigrations
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
python manage.py collectstatic
```

```
# Model
    class ModelName(models.Model):
    field = models.FieldType(options)
    def __str__(self):
```

```
        return self.field

# View
def view_name(request):
    data = Model.objects.all()
    return render(request, 'template.html', {'data': data})

# URL
path('route/', views.view_name, name='url_name')
```

End of Part 1

Next: Part 2 will cover Django REST Framework - Serializers, Function-Based API Views, and CRUD Operations

▼ Part 2

Part 2: Introduction to REST Framework

1. Introduction to APIs and REST

1.1 What is an API?

API (Application Programming Interface): A two-way communication bridge between frontend and backend.

- **Client:** Mobile app, React app, etc.
- **Server:** Django backend
- **API:** Middleman that carries requests and responses

Analogy: Restaurant – you (client) → waiter (API) → kitchen (server)

1.2 What is REST?

REST (Representational State Transfer): A set of principles for building APIs.

Key Principles:

- **Stateless:** Server doesn't store client state between requests
- **Client-Server:** Separation of concerns
- **Uniform Interface:** Standard methods (GET, POST, PUT, DELETE)

- **JSON/XML:** Standard data formats

1.3 HTTP Methods

Method	Purpose
GET	Retrieve data
POST	Create new data
PUT	Update existing data
DELETE	Delete data

1.4 JSON Format

```
{
    "id": 1,
    "name": "John Doe",
    "email": "john@example.com"
}
```

- Key-value pairs
- Double quotes required
- Similar to Python dictionary

---

2. Django REST Framework Setup

create students_main app, students app, api app. // for practice

2.1 Install DRF

```
pip install djangorestframework
```

2.2 Add to INSTALLED_APPS

```
INSTALLED_APPS = [    #students_main settings.py
    # ...
    'rest_framework',
]
```

---

3. Serializers

Serialization: Process of Converting model objects/querysets to JSON format.

Deserialization: Converting JSON back to model objects.

3.1 ModelSerializer

```
# api/serializers.py
from rest_framework import serializers
from students.models import Student

class StudentSerializer(serializers.ModelSerializer):
    class Meta:
        model = Student
        fields = '__all__'  # or list specific fields
        # fields = ['id', 'name, 'email']
```

3.2 Using Serializer

```
# Serialize a single object
serializer = StudentSerializer(employee)
serializer.data  # JSON-like dictionary

# Serialize multiple objects
serializer = StudentSerializer(employees, many=True)
serializer.data  # List of dictionaries
```

@api_view() rest decorator: it allow or deny the request

---

4. Function-Based API Views

4.1 URL Configuration

```
# project/urls.py
path('api/v1/', include('api.urls')),

# api/urls.py
from django.urls import path
```

```
from . import views

urlpatterns = [
    path('employees/', views.employees_view, name='employees'),
    path('students/<int:pk>/', views.StudentDetailView, name='StudentD
etailView'),
]
```

4.2 GET & POST (List & Create)

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from rest_framework import status
from students.models import Student
from .serializers import StudentSerializer

@api_view(['GET', 'POST'])
def StudentView(request):
    if request.method == 'GET':
        students= Student.objects.all()
        serializer = StudentSerializer(students, many=True)
        return Response(serializer.data, status=status.HTTP_200_OK)

    elif request.method == 'POST':
        serializer = StudentSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CR
EATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_
REQUEST)
```

4.3 GET, PUT, DELETE (Detail)

```
from django.shortcuts import get_object_or_404

@api_view(['GET', 'PUT', 'DELETE'])
def StudentDetailView(request, pk):
```

```
    student = get_object_or_404(Student, pk=pk)

    if request.method == 'GET':
        serializer = StudentSerializer(student)
        return Response(serializer.data, status=status.HTTP_200_OK)

    elif request.method == 'PUT':
        serializer = StudentSerializer(Student, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_200_O
K)
        return Response(serializer.errors, status=status.HTTP_400_BAD_
REQUEST)

    elif request.method == 'DELETE':
        Student.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)
```

Key Points:

- `@api_view` decorator required
- `request.data` for incoming data
- `serializer.data` for outgoing data
- Always return status codes

---

5. Class-Based API Views (APIView)

provide more structured and organized way to handle requests using object-oriented principles. no if/else needed, only get, post, put, delete

create employees app & its model with fields emp_id, name, designation.

5.1 List & Create

```
from rest_framework.views import APIView
from rest_framework.response import Response
from rest_framework import status
```

```
class EmployeesView(APIView):
    def get(self, request):
        employees = Employee.objects.all()
        serializer = EmployeeSerializer(employees, many=True)
        return Response(serializer.data, status=status.HTTP_200_OK)

    def post(self, request):
        serializer = EmployeeSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CR
EATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_
REQUEST)
```

5.2 Detail (GET, PUT, DELETE)

```
class EmployeeDetailView(APIView):
    def get_object(self, pk):
        return get_object_or_404(Employee, pk=pk)

    def get(self, request, pk):
        employee = self.get_object(pk)
        serializer = EmployeeSerializer(employee)
        return Response(serializer.data, status=status.HTTP_200_OK)

    def put(self, request, pk):
        employee = self.get_object(pk)
        serializer = EmployeeSerializer(employee, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_200_O
K)
        return Response(serializer.errors, status=status.HTTP_400_BAD_
REQUEST)

    def delete(self, request, pk):
            employee = self.get_object(pk)
```

```
        employee.delete()
        return Response(status=status.HTTP_204_NO_CONTENT)
```

5.3 URL Configuration

```
path('employees/', views.EmployeesView.as_view(), name='employees'),
path('employees/<int:pk>/', views.EmployeeDetailView.as_view(), name
='employee_detail'),
```

Note: Must use `.as_view()` for class-based views.

---

6. Mixins and Generics (use for common crud operations)

Mixins are reusable code classes in object-oriented programming that provide reusable built-in functionality for common CRUD operations. In RDF, mixins are used to add common functionality to views. i.e .list(), .create(), update(), delete().

6.1 Available Mixins

Mixin	Provides	what it do
`ListModelMixin`	`.list()`	list of objects
`CreateModelMixin`	`.create()`	
`RetrieveModelMixin`	`.retrieve()`	return single instance
`UpdateModelMixin`	`.update()`	
`DestroyModelMixin`	`.destroy()`	

6.2 GenericAPIView

mixins only work with GenericAPIView. generics will come with request methods i.e get, post, put, delete

class Employees(mixins, generics.GenericAPIView)

get( ), post( ), put ( ), delete( )

Provides:

- `queryset` attribute
- `serializer_class` attribute
- `get_object()` method

brief note:

> mixins will come with mixins classes and generics will come with request methods and provide views.

6.3 Example: List & Create

```
from rest_framework import mixins, generics

class EmployeesView(mixins.ListModelMixin,
                    mixins.CreateModelMixin,
                    generics.GenericAPIView):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer

    def get(self, request):
        return self.list(request)

    def post(self, request):
        return self.create(request)
```

6.4 Example: Detail (Retrieve, Update, Destroy)

```
class EmployeeDetailView(mixins.RetrieveModelMixin,
                         mixins.UpdateModelMixin,
                         mixins.DestroyModelMixin,
                         generics.GenericAPIView):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer

    def get(self, request, pk):
        return self.retrieve(request, pk)

    def put(self, request, pk):
        return self.update(request, pk)

    def delete(self, request, pk):
        return self.destroy(request, pk)
```

6.5 Even Shorter: Generic Classes

DRF provides pre-built generic classes that combine mixins:

ListAPIView —> For listing the objects

CreateAPIView —> For creating the objects

RetrieveAPIView —> For retrieving a single object using pk

UpdateAPIView —> For updating a single object using pk

DestroyAPIView —> For deleting an object using pk

ListCreateAPIView —> For listing and creating objects

RetrieveUpdateAPIView —> For retrieving & updating objects using pk

RetrieveUpdateDestroyAPIView —> For retrieving, updating & deleting objects using pk

```
from rest_framework import generics

# List + Create
class EmployeesView(generics.ListCreateAPIView):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer

# Retrieve + Update + Destroy
class EmployeeDetailView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer
    lookup_field = 'pk' #optional / if lookup_field are different name
```

URLs:

```
path('employees/', views.EmployeesView.as_view()),
path('employees/<int:pk>/', views.EmployeeDetailView.as_view()),
```

This is the most concise way to build CRUD APIs!

Recomemded: Functional Based API Views are recommended for complex api logic.

Summary Cheat Sheet

```
# Serializer
class MySerializer(serializers.ModelSerializer):
    class Meta:
        model = MyModel
        fields = '__all__'

# Function-Based API
@api_view(['GET', 'POST'])
def view_name(request):
    if request.method == 'GET':
        data = Model.objects.all()
        serializer = MySerializer(data, many=True)
        return Response(serializer.data, status=status.HTTP_200_OK)
    elif request.method == 'POST':
        serializer = MySerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CR
EATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_
REQUEST)

# Class-Based API
class MyView(APIView):
    def get(self, request):
        ...
    def post(self, request):
        ...
    def put(self, request, pk):
        ...
    def delete(self, request, pk):
        ...

# Generic (Shortest)
class MyListCreateView(generics.ListCreateAPIView):
    queryset = Model.objects.all()
    serializer_class = MySerializer
```

```python
class MyDetailView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Model.objects.all()
    serializer_class = MySerializer
```

End of Part 2

Next: Part 3 will cover advanced DRF topics: ViewSets, Routers, Authentication (JWT), Permissions in DRF, and Deployment.

▼ Part 3

Part 3: ViewSets, Routers & What's Next in DRF

> **Note:** The provided transcript ends at **Mixins & Generics**. Topics like ViewSets, Routers, JWT Authentication, and DRF Permissions are mentioned by the instructor as "coming next" but are **not fully taught** in this transcript. Below is a **conceptual overview** of those topics based on the instructor's mentions, so you have a complete picture. Full notes on these will require the next sessions.

1. Why ViewSets? (From the Instructor's Perspective)

The instructor repeatedly states:

> *"Next view sets will come — the view set will just act like magic. Only three lines of code, everything will be done."*

Goal of ViewSets:

- Combine all CRUD operations into a **single class**
- Reduce code from ~30 lines (mixins) to **~3 lines**
- Handle both list-level and detail-level operations in one place

2. ViewSets (Concept Overview)

2.1 What is a ViewSet?

A **ViewSet** is a class-based view that provides actions for CRUD operations without defining separate `get()`, `post()`, `put()`, `delete()` methods. Instead, it provides **actions** like:

Action	Maps to HTTP Method
`list()`	GET (all)
`create()`	POST
`retrieve()`	GET (single)
`update()`	PUT
`partial_update()`	PATCH
`destroy()`	DELETE

```
#api/views.py   viewset: allow write all method in single class and
# using router you don't need to write all endpoints url in urls.py
cass EmployeeViewSet(viewsets.ViewSet):
    def list(self, request):
        employees = Employee.objects.all()
        serializer = EmployeeSerializer(employees, many=True)
        return Response(serializer.data)

    def create(self, request):
        serializer = EmployeeSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CR
EATED)
        return Response(serializer.errors)

    def retrieve(self, request, pk):
        employee = get_object_or_404(Employee, pk=pk)
        serializer = EmployeeSerializer(employee)
        return Response(serializer.data, status=status.HTTP_200_OK)

    def update(self, request, pk):
        employee = get_object_or_404(Employee, pk=pk)
        serializer = EmployeeSerializer(employee, data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_200_O
K)
```

2.2 ModelViewSet (Most Common)

```
from rest_framework import viewsets

class EmployeeViewSet(viewsets.ModelViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer
```

That's it — 3 lines. This handles GET, POST, PUT, PATCH, DELETE automatically.

2.3 ReadOnlyModelViewSet

For read-only APIs (only GET):

```
class EmployeeReadOnlyViewSet(viewsets.ReadOnlyModelViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer
```

---

3. Routers (Concept Overview)

When using ViewSets, you don't write `path()` manually. Instead, you use a **Router** that auto-generates URLs.

3.1 DefaultRouter

```
from rest_framework.routers import DefaultRouter
from .views import EmployeeViewSet

router = DefaultRouter()
router.register('employees', EmployeeViewSet, basename='employees')

urlpatterns = router.urls
```

3.2 Auto-Generated URLs

URL	Method	Action
`/employees/`	GET	list
`/employees/`	POST	create

URL	Method	Action
`/employees/<pk>/`	GET	retrieve
`/employees/<pk>/`	PUT	update
`/employees/<pk>/`	PATCH	partial_update
`/employees/<pk>/`	DELETE	destroy

No manual URL patterns needed!

---

4. Comparison: All Approaches Side-by-Side

Approach	Lines of Code	Control	Best For
Function-Based	~30	Full	Custom logic, complex business rules
APIView	~25	Full	Custom logic with class organization
Mixins + Generics	~15	Moderate	Common CRUD with some customization
Generic Classes	~5	Low	Standard CRUD
ViewSets + Routers	~3	Lowest	Standard CRUD, quick APIs

Instructor's Advice:

> *"As a senior developer, mostly we choose mixins... If logic is very complex, use APIView. If standard CRUD, use ViewSets."*

---

5. JWT Authentication (Concept Overview)

5.1 What is JWT?

JWT (JSON Web Token): A token-based authentication system for APIs.

- **Access Token:** Short-lived (e.g., 5–15 min)
- **Refresh Token:** Long-lived (e.g., 7 days)

5.2 Flow

```
1. User logs in with username/password
2. Server returns Access Token + Refresh Token
3. Client stores tokens
4. Client sends Access Token in Authorization header for each request
5. When Access Token expires, client uses Refresh Token to get new Access Token
```

5.3 Header Format

```
Authorization: Bearer <access_token>
```

5.4 hands-on guide for jwt authentication

1. Installation

Install the package using `pip`. If you plan to use digital signature algorithms like RSA or ECDSA, install it with the `crypto` extra:

```
pip install djangorestframework-simplejwt[crypto]
```

2. Configure DRF Settings

Register the plugin as a default authentication class in your `settings.py` file:

```
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    )
}
```

3. Set Up Route Endpoints

Map out the token routes in your project's main `urls.py` file to handle log-ins and token life cycles.

```
from django.urls import path
from rest_framework_simplejwt.views import (
    TokenObtainPairView,
    TokenRefreshView,
    TokenVerifyView,
)

urlpatterns = [
    # Login endpoint: Sends username/password -> returns Access & Refr
esh tokens
    path('api/token/', TokenObtainPairView.as_view(), name='token_obta
in_pair'),
```

```
    # Refresh endpoint: Sends Refresh token -> returns new Access toke
n
    path('api/token/refresh/', TokenRefreshView.as_view(), name='token
_refresh'),

    # Optional verification endpoint: Validates an existing token
    path('api/token/verify/', TokenVerifyView.as_view(), name='token_v
erify'),
]
```

⚙️ Common Configuration Setting

You can customize token behavior by defining a `SIMPLE_JWT` block in your `settings.py`. Here is a production-safe baseline.

```
from datetime import timedelta

SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(minutes=15),   # Short lifespan
for security
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7), # Longer lifespan to
stay loggedin
    'ROTATE_REFRESH_TOKENS': True,       # Issue a new refresh token o
n every usage
    'BLACKLIST_AFTER_ROTATION': True,   # Destroys the old refresh tok
en (Requires blacklist app)

    'ALGORITHM': 'HS256',
    'AUTH_HEADER_TYPES': ('Bearer',)#Expected header: Authorization: B
earer <token>
}
```

🔒 Core Features

- **Stateless Authentication:** Server-side sessions are not needed, lowering server resource consumption.
- **Token Rotation & Blacklisting:** Enables you to safely invalidate stolen tokens immediately by installing the built-in `rest_framework_simplejwt.token_blacklist` app.

- **Custom Claims:** Allows you to effortlessly inject custom user attributes (like `is_staff` or `role`) directly into the encoded token payload.

---

6. DRF Permissions (Concept Overview)

6.1 Built-in Permission Classes

```
from rest_framework.permissions import (
    AllowAny, IsAuthenticated, IsAdminUser, IsAuthenticatedOrReadOnly
)
```

6.2 Usage in Views

```
class EmployeeViewSet(viewsets.ModelViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer
    permission_classes = [IsAuthenticated]
```

6.3 Custom Permissions

```
from rest_framework.permissions import BasePermission

class IsOwnerOrReadOnly(BasePermission):
    def has_object_permission(self, request, view, obj):
        if request.method in ['GET', 'HEAD', 'OPTIONS']:
            return True
        return obj.owner == request.user
```

---

7. Pagination (Concept Overview)

7.1 Settings.py

```
REST_FRAMEWORK = {
    'DEFAULT_PAGINATION_CLASS': 'rest_framework.pagination.PageNumberP
agination',
```

```
    'PAGE_SIZE': 10,
}
```

7.2 Response Format

```
{
    "count": 100,
    "next": "<http://api.example.com/employees/?page=2>",
    "previous": null,
    "results": [...]
}
```

8. Filtering, Searching, Ordering

8.1 Install

```
pip install django-filter
```

8.2 Settings.py

```
INSTALLED_APPS = [
        'django_filters',
]

REST_FRAMEWORK = {
    'DEFAULT_FILTER_BACKENDS': [
        'django_filters.rest_framework.DjangoFilterBackend',
        'rest_framework.filters.SearchFilter',
        'rest_framework.filters.OrderingFilter',
    ],
}
```

8.3 Usage

```
from django_filters.rest_framework import DjangoFilterBackend
from rest_framework.filters import SearchFilter
```

```python
class EmployeeViewSet(viewsets.ModelViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer
    filterset_fields = ['designation']
    filter_backends = [SearchFilter, DjangoFilterBackend]
    search_fields = ['emp_name']
```

9. Throttling (Rate Limiting)

9.1 Settings

```python
REST_FRAMEWORK = {
    'DEFAULT_THROTTLE_CLASSES': [
        'rest_framework.throttling.AnonRateThrottle',
        'rest_framework.throttling.UserRateThrottle',
    ],
    'DEFAULT_THROTTLE_RATES': {
        'anon': '100/day',
        'user': '1000/day',
    },
}
```

10. Custom User Model (Mentioned in Transcript)

The instructor mentions:

> *"If you want to add mobile number or change login from username to email, extend the user model. I have a video on this."*

10.1 Why Custom User Model?

- Default user uses **username** for login
- Many apps want **email** as login
- May need extra fields (phone, address, etc.)

10.2 Quick Example

```python
from django.contrib.auth.models import AbstractUser

class CustomUser(AbstractUser):
    email = models.EmailField(unique=True)
    phone = models.CharField(max_length=15, blank=True)

    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = ['username']
```

10.3 Settings

```python
AUTH_USER_MODEL = 'accounts.CustomUser'
```

Important: Must be set **before** running migrations.

---

11. Full Project Flow (Recap)

```
1. Create virtual environment
2. Install Django + DRF
3. Create project + app(s)
4. Configure settings.py
5. Create models
6. makemigrations + migrate
7. Register in admin
8. Create serializer
9. Create URL patterns
10. Create views (function → class → mixins → generics → viewsets)
11. Test with Postman / Browsable API
12. Add authentication (JWT)
13. Add permissions
14. Add pagination, filtering
15. Deploy
```

---

12. What the Instructor Says Comes Next

From the transcript:

Week	Topic
Week 1–2	Django basics, models, views, templates
Week 3	DRF — serializers, function-based APIs, class-based views, mixins, generics

Week	Topic
Week 4+	ViewSets, Routers, JWT, Permissions, Real Project
Later	E-commerce project, Docker, CI/CD, GitHub Actions, AWS S3, Deployment

13. Key Takeaways from the Entire Course

13.1 Django Core

- MVT architecture — Model never talks to Template directly
- Always use virtual environments
- Migrations are auto-generated blueprints for the database
- `get_object_or_404` handles missing objects gracefully
- `@login_required` protects views
- Context processors provide `user` to all templates

13.2 DRF

- **Serialization** = model → JSON
- **Deserialization** = JSON → model
- Always return proper **HTTP status codes**
- Use `many=True` when serializing querysets
- `request.data` = incoming data
- `serializer.data` = outgoing data
- `@api_view` decorator is **mandatory** for function-based API views
- Use `.as_view()` for class-based views in URLs

13.3 Choosing the Right Approach

Situation	Use
Complex custom logic	Function-Based or APIView
Standard CRUD with some logic	Mixins + Generics
Pure standard CRUD	Generic Classes or ViewSets
Quick prototype	ViewSets + Routers

13.4 Instructor's Golden Rule

> *"Don't just watch this course. Code along with me to get the most out of it."*
>
> *"If the logic is very complex, do not try to fit everything inside mixins. Use function-based views for full control."*

14. Final Code Cheat Sheet

```
# ============ MODELS ============
class Employee(models.Model):
    first_name = models.CharField(max_length=50)
    last_name = models.CharField(max_length=50)
    email = models.EmailField(unique=True)
    salary = models.IntegerField()
    joining_date = models.DateField()
    is_active = models.BooleanField(default=True)
    department = models.ForeignKey('Department', on_delete=models.SET_
NULL, null=True)
    manager = models.ForeignKey('self', on_delete=models.SET_NULL, nul
l=True, blank=True)

    def __str__(self):
        return f"{self.first_name} {self.last_name}"

# ============ SERIALIZER ============
class EmployeeSerializer(serializers.ModelSerializer):
    class Meta:
        model = Employee
        fields = '__all__'

# ============ FUNCTION-BASED VIEW ============
@api_view(['GET', 'POST'])
def employees_view(request):
    if request.method == 'GET':
        employees = Employee.objects.all()
        serializer = EmployeeSerializer(employees, many=True)
        return Response(serializer.data, status=status.HTTP_200_OK)
    elif request.method == 'POST':
```

```
        serializer = EmployeeSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CR
EATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_
REQUEST)

# ============ API VIEW (Class-Based) ============
class EmployeesView(APIView):
    def get(self, request):
        employees = Employee.objects.all()
        serializer = EmployeeSerializer(employees, many=True)
        return Response(serializer.data)

    def post(self, request):
        serializer = EmployeeSerializer(data=request.data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data, status=status.HTTP_201_CR
EATED)
        return Response(serializer.errors, status=status.HTTP_400_BAD_
REQUEST)

# ============ GENERIC (Shortest for CRUD) ============
class EmployeesView(generics.ListCreateAPIView):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer

class EmployeeDetailView(generics.RetrieveUpdateDestroyAPIView):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer

# ============ VIEWSET (Most Concise) ============
class EmployeeViewSet(viewsets.ModelViewSet):
    queryset = Employee.objects.all()
    serializer_class = EmployeeSerializer

# ============ ROUTER ============
```

```
router = DefaultRouter()
router.register('employees', EmployeeViewSet, basename='employees')
urlpatterns = router.urls

# ============ URLs ============
path('employees/', views.EmployeesView.as_view()),
path('employees/<int:pk>/', views.EmployeeDetailView.as_view()),
```

15. HTTP Status Codes Reference

Code	Meaning	When to Use
200	OK	GET success, PUT success
201	Created	POST success
204	No Content	DELETE success
400	Bad Request	Validation errors
401	Unauthorized	Not logged in
403	Forbidden	Logged in but no permission
404	Not Found	Object doesn't exist
500	Internal Server Error	Server-side bug

End of Part 3 — and End of Complete Course Notes

Summary of All Parts

Part	Topics
Part 1	Django fundamentals, MVT, models, migrations, admin, templates, static/media, forms, auth basics
Part 2	Authentication, permissions, groups, DRF setup, serializers, function-based APIs, class-based APIs, mixins, generics
Part 3	ViewSets, Routers, JWT, DRF permissions, pagination, filtering, custom user model, full project flow

Revision Tip: Practice by rebuilding the Employee/Student project from scratch without looking at notes. Then try a new model (e.g., Books, Products) using all approaches — function-based, APIView, mixins, generics, and ViewSets.